Today A Reader,
Tomorrow A Leader

Dezyne École College
www.dezyneecole.com

RESILIO

Smart Hospitality. Resilient Operations

RESILIENCE REPORT

Project Type: Resilient Restaurant Management System.

Technology Stack: React.js, Node.js, Express.js, MongoDB, Python.

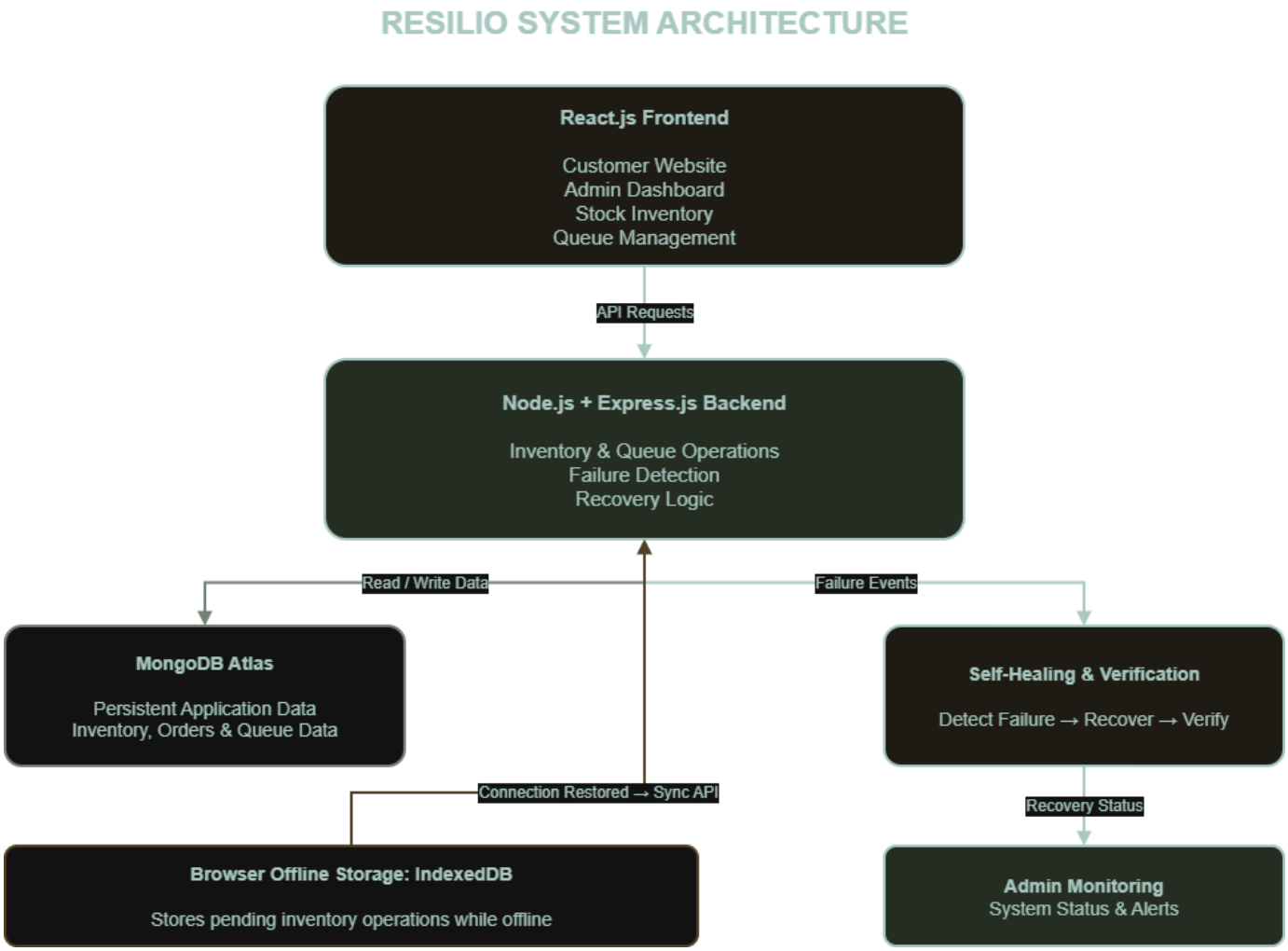
Key Modules: Stock Inventory Management, Self-Healing Recovery, Queue Management.

1.Project Overview

RESILIO is a resilient restaurant management system designed to improve the reliability of restaurant operations by detecting and recovering from selected system failures.

The system integrates stock inventory management, offline inventory recovery, expired stock detection, and queue management. Its self-healing mechanisms aim to reduce manual intervention, improve data consistency, and restore normal operations after recoverable failures.

2. System Architecture



System Architecture Description

RESILIO follows a client-server architecture using React.js for the frontend, Node.js and Express.js for backend operations, and MongoDB Atlas for data storage. It includes Stock Inventory and Queue Management modules. The self-healing mechanisms detect failures, initiate recovery, and verify system status. IndexedDB stores pending inventory operations during network interruptions and synchronizes them with the backend when the connection is restored.

3. Before-and-After Comparison

The following table compares how RESILIO handles failures before and after implementing self-healing mechanisms.

Metric	Before Self-Healing	After Self-Healing
Offline Inventory Recovery	Inventory operations could require manual re-entry after a network interruption.	Pending inventory operations are stored in IndexedDB and synchronized with the backend when connectivity is restored.
Expired Stock Detection	Staff might need to check stock expiry dates manually.	The system identifies expired batches and can mark them as expired and notify the administrator.
Duplicate Token Failure	Repeated requests could potentially create duplicate queue tokens.	Request validation and duplicate checks can help prevent duplicate token creation.
Queue Status Synchronisation	The displayed queue status might become outdated and require manual refreshing.	Queue information can be refreshed from the backend to restore consistency.

4. Failure Testing and Recovery Time

The following table describes the four failure scenarios considered in RESILIO, their detection methods, recovery mechanisms, and measured recovery times.

Failure Scenario	How It Is Detected	How It Heals	Measured Recovery Time
1. Offline Inventory Recovery	The application detects network disconnection using browser connectivity events.	Pending inventory operations are stored in IndexedDB and synchronized with the backend when the connection is restored.	15 seconds
2. Expired Stock Detection	The system checks stock batch expiry dates and identifies expired batches.	Expired batches are marked as EXPIRED, their remaining quantity is removed from available stock, and an admin alert is generated.	6 seconds
3. Duplicate Token Failure	The system checks for repeated token-generation requests or duplicate tokens.	Duplicate token creation is prevented through request validation and duplicate checks.	4 seconds
4. Queue Status Synchronisation Failure	The system identifies inconsistent queue statuses between the backend and the user interface.	Queue information is refreshed from the backend to restore consistency.	3 seconds

5. Known Limitations

1 Offline Inventory Limitations

Offline operations depend on browser storage and network restoration. Browser data loss, storage restrictions, or persistent backend failures may prevent successful synchronization.

2 Expired Stock Limitations

Expiry detection depends on accurate expiry dates and batch records. Incorrect input data may lead to incorrect stock availability decisions.

3 Duplicate Token Prevention Limitations

Duplicate token prevention depends on backend validation and reliable request identification. Without idempotency protection or appropriate database constraints, repeated requests may still create duplicate tokens.

4 Queue Status Synchronisation Limitations

Queue status synchronization depends on database availability and the application's ability to retrieve the latest valid state. Network delays and concurrent updates may cause temporary inconsistencies.

5 Recovery Monitoring Limitations

Recovery time measurements depend on reliable timestamps and clear completion criteria. A successful API response alone may not prove that the final inventory or queue state is correct.

Submitted By

Dakshi Dhiman & Deepika Sharma